Autonomous Enterprise Decision Intelligence Using Agentic AI

Ananya Naga Raj
Dept. of Computer Science
Syracuse University
Syracuse, NY, USA
annagara@syr.edu

Abhijnya Konanduru Gurumuthy
Dept. of Computer Science
Syracuse University
Syracuse, NY, USA
akonandu@syr.edu

Amulya Naga Raj
Dept. of Computer Science
Syracuse University
Syracuse, NY, USA
amnagara@syr.edu

Sunil Hanumanthegowda Kote
Dept. of Computer Science
Syracuse University
Syracuse, NY, USA
sukote@syr.edu

***Abstract*—This project presents an autonomous enterprise decision intelligence system that processes structured time-series business metrics and generates outputs from anomaly detection to actionable recommendations. The system operates on key indicators such as revenue, conversion rate, churn signals, and customer acquisition cost derived from the Google Analytics 4 dataset. A coordinated multi-agent architecture is implemented using five agents monitoring, anomaly detection, root cause analysis, decision making, and explanation supported by a Central Controller and Shared Memory for structured coordination.**

The monitoring agent maintains rolling statistical baselines, while the anomaly detection agent identifies deviations and assigns confidence. The root cause analysis agent computes contribution scores across dimensions such as region, product tier, and channel. The decision agent ranks corrective actions using a utility function based on impact, risk, and cost. The explanation agent produces structured summaries from computed outputs.

The system operates through a sequential think-act-observe cycle. Experimental evaluation under a balanced sensitivity configuration achieves an F1 score of 0.50, outperforming baseline methods while achieving a better balance between precision and recall compared to static and rule-based approaches.

***Index Terms*—Agentic AI, Anomaly Detection, Root Cause Analysis, Decision Intelligence, Multi Agent Systems, Time Series Analysis**

I. Introduction

Enterprise systems continuously generate structured business metrics such as revenue, conversion rate, churn signals, and customer acquisition cost across dimensions including region, product tier, customer segment, and channel. These metrics are typically monitored using dashboards that highlight trends and threshold-based alerts. However, when abnormal changes occur, the process of identifying causes and deciding corrective actions remains manual. Analysts must examine multiple dimensions, compare historical values, and determine responses without a unified framework, which increases response time and introduces inconsistency.

This project addresses this gap by designing an autonomous enterprise decision intelligence system that produces a complete decision flow from anomaly detection to action recommendation. The system processes structured time series data derived from the Google Analytics 4 dataset and operates through five specialized agents. The monitoring agent computes rolling mean, rolling standard deviation, and exponentially weighted moving average for each metric. The anomaly detection agent identifies deviations using a standardized score and assigns confidence value. The root cause analysis agent computes contribution scores across dimensions to identify primary drivers. The decision agent ranks predefined corrective actions using a utility score based on expected impact, risk, and operational cost. The explanation agent generates structured summaries from computed outputs.

A Central Controller governs the execution sequence of agents, enforces confidence thresholds, and triggers reevaluation when inconsistencies are detected. All agents interact through a Shared Memory structure that maintains intermediate states including rolling statistics, anomaly scores, root cause rankings, and action outputs. Each agent operates as a goal-oriented component with a defined reasoning function, and the controller acts as the planning layer that determines progression, reevaluation, or halting based on intermediate outputs. The system follows a perception, reasoning, action, and feedback cycle where rolling baselines are updated after each processing step.

The objective of this work is to evaluate whether a coordinated multiagent system improves anomaly detection consistency, root cause identification, and decision quality compared to non-agentic baselines. The system is evaluated using three baselines including a static threshold method, a z score only model, and a manual aggregation approach. Performance is measured using precision, recall, F1 score, detection latency, root cause accuracy, and utility-based evaluation.

The remainder of this report is organized as follows. Section II reviews related work in anomaly detection, root cause analysis, and multi agent systems. Section III describes methodology and system architecture. Section IV presents the implementation details. Section V reports the evaluation results. Section VI discusses observations and limitations. Section VII concludes the work and outlines future directions.

II. Literature Review

Anomaly detection in time series data has been widely studied in the context of enterprise monitoring and financial systems [1], [2], [3]. Statistical methods such as moving averages, rolling standard deviation, and threshold-based detection are commonly used due to their interpretability and low computational cost. More advanced approaches include

machine learning models that capture seasonality and nonlinear patterns. However, most existing methods focus only on identifying deviations and do not extend to structured diagnosis or decision support. As a result, anomaly detection systems typically generate alerts that require manual investigation.

Root cause analysis methods aim to identify the factors contributing to observed changes in business metrics. Contribution based attribution techniques compute the share of deviation across different dimensions such as region, product, or channel. These methods are grounded in causal inference and attribution frameworks [4]. However, they are often applied as standalone analytical tools and are not integrated into an automated decision pipeline. In practice, analysts must interpret these outputs and determine corrective actions without system level coordination.

Decision support systems in enterprise environments often rely on predefined rules or manual workflows. While some systems incorporate optimization or scoring mechanisms, they typically operate independently of anomaly detection and root cause analysis components. This separation leads to fragmented workflows where detection, diagnosis, and action selection are handled as independent stages without shared state or coordinated control.

Recent work on multi agent systems has explored distributed reasoning and coordinated decision making [5]. In parallel, large language model-based agent frameworks have introduced approaches for combining reasoning and action. Methods such as explainable AI focus on improving transparency and interpretability of system outputs [6]. Approaches such as ReAct demonstrate how agents can interleave reasoning and action through iterative interaction with tools [7]. Tool use frameworks such as Toolformer [8] and multi agent communication frameworks further extend this idea by enabling structured interaction between components [9]. However, many implementations emphasize flexible task execution rather than controlled coordination. These systems often lack explicit mechanisms for validating intermediate outputs, managing shared state, or enforcing progression conditions.

The key limitation across existing work is the lack of integration between detection, diagnosis, and action within a coordinated system. Most approaches either focus on a single stage or treat each stage independently without shared memory or control logic. This project addresses these limitations by implementing a structured multi agent system where all stages operate within a single coordinated pipeline using a Central Controller and Shared Memory. The system combines statistical anomaly detection, contribution-based root cause analysis, and utility-based decision ranking into a unified framework with controlled execution and reproducible outputs.

III. Methodology

A. System Design and Architecture

The system is designed as a coordinated multi agent architecture that processes structured time series business metrics and produces decision outputs through a controlled workflow. The architecture consists of five agents, a Central Controller, and a Shared Memory structure. The system operates in a perception, reasoning, action, and feedback cycle where each stage writes its output to Shared Memory and the controller governs progression.

The Central Controller enforces execution order and validates intermediate outputs. It checks confidence thresholds before allowing progression from anomaly detection to root cause analysis and decision stages. If inconsistencies are detected such as low confidence or missing data, the controller triggers re evaluation or halts execution. This prevents unreliable outputs from propagating across stages. When downstream outputs conflict with upstream confidence levels, the controller initiates re-evaluation by running a stricter anomaly detection pass before allowing the pipeline to proceed. This conflict resolution mechanism ensures that only validated and consistent results reach the decision and explanation stages.

Shared Memory is implemented as a structured state object that stores timestamp, metric name, current value, rolling mean, rolling standard deviation, anomaly score, confidence value, ranked root causes, and ranked actions. All agents read from and write to this shared state, ensuring consistent coordination. The controller validates that required fields are populated at each stage before allowing progression, preventing incomplete or inconsistent outputs from propagating downstream.

Each agent operates as a goal-oriented component with a defined reasoning function, and the controller acts as the planning layer that determines progression, reevaluation, or halting based on intermediate outputs.

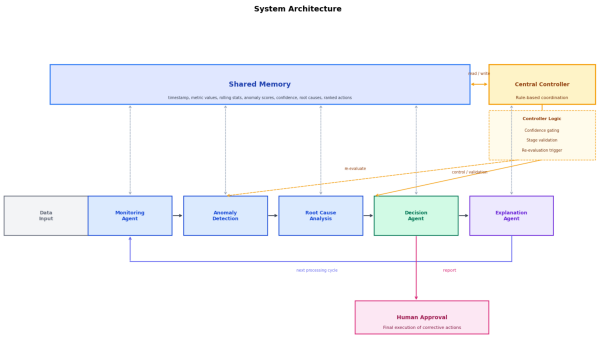

Fig. 1. System architecture showing multi agent workflow with Central Controller and Shared Memory.

B. Agent Architecture and Reasoning Process

The system consists of five agents with defined responsibilities.

The monitoring agent ingests time series data and computes rolling mean, rolling standard deviation, and exponentially weighted moving average for each metric. These statistics are written to Shared Memory and serve as the baseline reference for all downstream agents.

The anomaly detection agent computes a standardized deviation score using the difference between the current value and rolling mean normalized by rolling standard deviation. The

score is combined with an exponentially weighted component to reduce noise sensitivity. A confidence value is assigned, and anomalies are flagged when the score exceeds a threshold. The system supports multiple sensitivity configurations: standard uses a z-score threshold of 2.5, balanced uses 2.0, sensitive uses 1.7, and explorer uses 1.45. The balanced configuration with threshold 2.0 is used for all reported evaluations. This threshold corresponds to the 95.4th percentile of a normal distribution and was selected from the predefined sensitivity presets based on evaluation performance, as it produced the strongest tradeoff between precision and recall on the GA4 dataset compared to stricter and more permissive alternatives.

The root cause analysis agent computes contribution scores across dimensions using statistical attribution. For each business dimension such as region, product tier, customer segment, and channel, the agent calculates the absolute deviation of the current value from its rolling baseline. Each dimension's deviation is then normalized against the total observed deviation to produce a contribution percentage. Dimensions are ranked by their contribution scores, and when multiple dimensions exhibit comparable scores, all significant contributors are reported with percentage attribution rather than forcing a single dominant cause. This approach ensures quantitative, interpretable, and reproducible attribution grounded entirely in observed metric differences without relying on causal inference modeling or LLM-based reasoning.

The decision agent evaluates predefined corrective actions using a utility-based scoring function. The utility function combines three components: expected impact derived from historical metric recovery rates following similar deviations, risk estimated as the variance of outcomes across past instances, and operational cost assigned based on predefined effort ratings. Each action receives a weighted utility score computed as: impact weight multiplied by expected impact, minus risk weight multiplied by risk variance, minus cost weight multiplied by normalized operational cost. Actions are ranked based on computed utility scores, and when utility differences between the top actions are marginal, ranked alternatives are presented rather than a single deterministic recommendation. This decision-theoretic framework ensures that recommendations are measurable, transparent, and consistently reproducible.

The explanation agent generates structured summaries using computed outputs. It receives a precomputed payload containing anomaly scores, contribution scores, and ranked actions, and produces a formatted report without performing independent numerical reasoning. This ensures consistency with upstream results.

C. Data Engineering and Dataset

The system operates on structured time series data derived from the Google Analytics 4 Obfuscated E-commerce Public Dataset available through Google BigQuery. Raw event level data is aggregated into daily metrics. Key performance indicators include revenue, conversion rate, churn signals, and customer acquisition cost. Data is organized across regions, product tier, customer segment, and channel. The pipeline processes data sequentially to simulate streaming input. Rolling statistics are updated at each step using only past observations, preventing data leakage.

D. Tool Integration and Implementation

The system is implemented in Python. Data processing and aggregation are performed using pandas. Statistical computations for rolling mean, standard deviation, and deviation scoring are implemented using numerical functions. The explanation agent optionally uses a language model API only for formatting outputs. No retrieval systems or external knowledge sources are used. All computations are based on structured numerical data.

E. Orchestration and Workflow

The Central Controller manages the execution flow. The monitoring agent writes rolling statistics to Shared Memory. The anomaly detection agent computes anomaly score and confidence. If confidence exceeds the threshold, the controller allows progression to root cause analysis. The root cause agent computes contribution scores. The decision agent ranks actions based on utility. The explanation agent generates the final output.

If confidence is below the threshold, the system flags the event and stops further processing. If required data is missing, the system falls back to the last valid baseline stored in Shared Memory. If conflicting outputs are detected, such as ranked actions produced without corresponding root causes, the controller initiates re-evaluation by running a stricter anomaly detection pass before producing the final output.

F. Safety Guardrails

The system includes safeguards to ensure reliable operation. Confidence based gating prevents low confidence outputs from propagating to downstream stages. The controller halts execution when required data is missing or when outputs conflict. Events with insufficient confidence are flagged instead of producing decisions. All recommended actions require human approval before execution.

G. Design Justification

A rule based Central Controller is used to ensure deterministic behavior and reproducibility. This avoids uncontrolled execution and ensures consistent outputs. Shared Memory provides a single coordinated state across all agents.

Statistical methods are used for anomaly detection and root cause analysis to maintain interpretability. Utility based scoring is used for decision ranking to evaluate actions using measurable criteria. This design produces consistent and traceable outputs suitable for enterprise decision support.

IV. Implementation

A. Agent Architecture and Components Configuration

The system is implemented as a modular multi agent pipeline where each agent performs a specific function in

the decision workflow. The Monitoring Agent processes time series data and computes rolling statistics including mean, standard deviation, and EWMA. The Anomaly Detection Agent evaluates deviations using a combined z score and EWMA based approach to generate anomaly scores and confidence values. The Root Cause Analysis Agent performs attribution by ranking contributing dimensions based on their proportional impact. The Decision Agent generates corrective actions ranked using a utility function based on impact, risk, and cost.

The Explanation Agent generates structured outputs using a controlled prompt design. The LLM receives a precomputed JSON payload containing anomaly scores, contribution scores, and ranked actions. It is instructed to produce three sections: what happened, why it happened, and what to do, without introducing or modifying numerical values. This ensures consistency and prevents hallucination.

Each agent operates independently while sharing state through a centralized memory, enabling consistent reasoning across the pipeline.

Agent	Input	Output
Monitoring Agent	Raw time series data	Rolling mean, std, EWMA
Anomaly Detection	Current value + baseline	Anomaly score, confidence
Root Cause Analysis	Anomaly + history	Ranked dimension contributions
Decision Agent	Root causes	Ranked actions
Explanation Agent	All outputs	Final explanation report

Table I: Agent Responsibilities

B. Memory Architecture

A shared memory structure is implemented to maintain state across all agents. It stores intermediate outputs such as rolling statistics, anomaly scores, attribution results, and decision rankings. This enables agents to operate with full contextual awareness of prior computations and ensures consistency across the pipeline.

Component	Description
Timestamp	Current time step
KPI value	Observed metric
Rolling stats	Mean, std, EWMA
Anomaly score	Deviation measure
Confidence	Detection certainty
Root causes	Ranked contributors
Actions	Ranked decisions

Table II: Shared Memory Contents

C. Tools Integration and Action Layer

The action layer is implemented using rule-based decision logic. The Decision Agent maps detected anomalies and root causes to corrective actions using predefined strategies. Each action is scored using a utility function that considers expected impact, risk, and cost.

Examples include campaign adjustments, promotional actions, retargeting strategies, and technical optimizations. Outputs are structured for execution while maintaining interpretability.

Safety is enforced through confidence gating and validation checks. Low confidence outputs are filtered, and the controller halts execution when inconsistencies are detected. All actions require human approval before execution.

D. Orchestration and Workflow

The system follows a sequential think act observe pipeline. Data flows from monitoring to anomaly detection, followed by root cause analysis and decision generation. A central controller coordinates execution by validating outputs, triggering reevaluation when inconsistencies are detected, and managing agent interactions.

The pipeline replays 92 daily records sequentially. At each time step, the controller executes one full cycle where the monitoring agent observes new data, agents perform reasoning using shared state, actions are generated, and rolling baselines are updated before the next step.

This event driven loop enables continuous adaptation of the system to evolving data patterns.

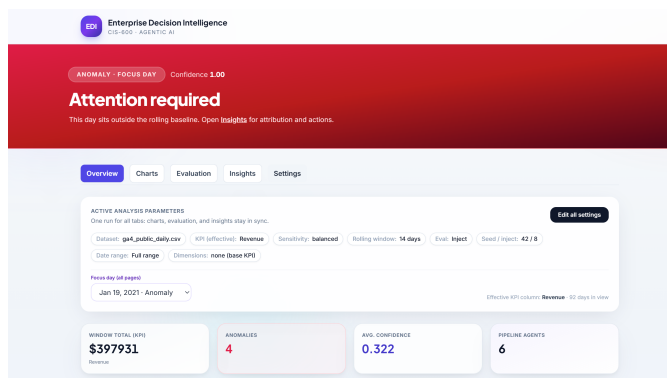

Fig. 2. Dashboard showing anomaly detection with confidence score and system level monitoring outputs.

V. Results

A. Detection Performance

The proposed agentic system was evaluated under a balanced sensitivity configuration, which provides an effective tradeoff between precision and recall. The evaluation uses two scenarios: controlled anomaly injection with known timestamps and ground truth root cause keys, and naturally occurring variations present in the GA4 dataset. Controlled injection allows precise measurement of detection accuracy and root cause precision, while natural variation testing confirms system robustness under realistic operating conditions.

The agentic system achieved an F1 score of 0.50, outperforming baseline approaches. Static thresholding achieved 0.17, z score-based detection achieved 0.22, and manual percentage change-based detection achieved 0.16. This represents an improvement of more than 2x over the best performing

Root causes

Dimension	Value	Share
product_tier	premium	26.29%
channel	referral	17.97%
region	Canada	14.19%
customer_segment	returning	11.34%
customer_segment	new	11.34%
product_tier	value	3.61%
region	United_States	3.57%
channel	data_deleted	2.27%
channel	organic	1.95%
region	India	1.71%
region	United_Kingdom	1.65%
region	Spain	1.4%
channel	none	0.68%
channel	cpc	0.61%
region	China	0.59%
region	France	0.29%
region	Japan	0.27%
channel	Other	0.27%

Recommended actions

Short promotional push on underperforming channel Impact / risk / cost	0.4953
Regional campaign adjustment Impact / risk / cost	0.4396
Retargeting / remarketing to high-intent segments Impact / risk / cost	0.4067
Refresh creatives / landing pages for underperforming traffic Impact / risk / cost	0.3925
Shopping feed & search listing hygiene (titles, GTIN, negatives) Impact / risk / cost	0.3724
Funnel / checkout experience audit Impact / risk / cost	0.3653
CRM / email push to recover stalled carts or churn risk Impact / risk / cost	0.3606
Rebalance inventory / product tier focus Impact / risk / cost	0.3278
Technical review: latency, errors, mobile UX on key paths Impact / risk / cost	0.3238
Limited-time price or bundle test on affected SKUs Impact / risk / cost	0.3087
Local or affiliate partnership push in stressed regions Impact / risk / cost	0.2848
Hold spend and monitor (low intervention) Impact / risk / cost	0.2308

Fig. 3. Root cause attribution and ranked corrective actions generated by the system.

Explanation

Revenue · Jan 19, 2021: value 10,704.60, baseline 2,637.65, spread 2,481.09.
Anomaly score 3.10 · confidence 1.00.
Top dimensional shifts: product_tier=premium (26.29%), channel=referral (17.97%), region=Canada (14.19%), customer_segment=returning (11.34%), customer_segment=new (11.34%).
Top action: Short promotional push on underperforming channel (utility 0.50) — historical_recovery_impact=0.335; next: Regional campaign adjustment.
Awaiting human approval before executing ranked actions.

Fig. 4. Generated insight summarizing anomaly, root causes, and recommended actions.

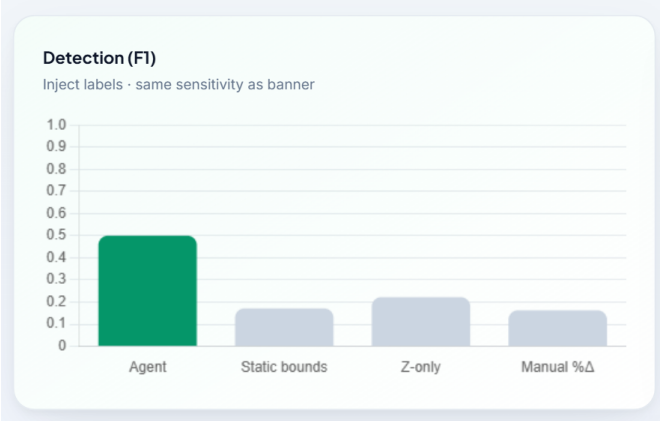

Fig. 5. Comparison of F1 score between the agentic system and baseline methods.

baseline, demonstrating the effectiveness of combining statistical deviation with contextual reasoning.

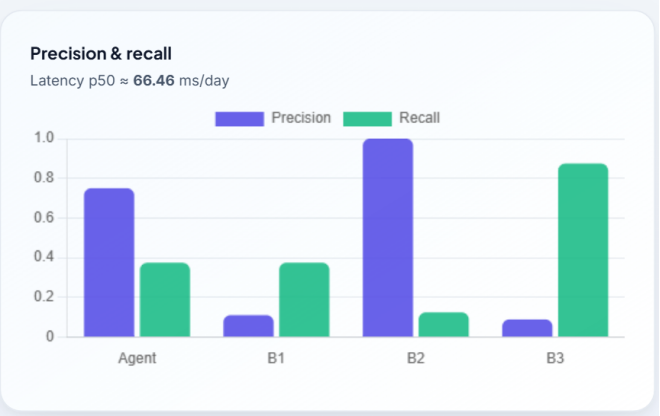

Fig. 6. Precision and recall comparison across agentic system and baseline methods.

The agentic system achieved a precision of 0.75 and a recall of 0.375, indicating that detected anomalies are highly accurate while still capturing a meaningful portion of true anomalies. In contrast, baseline methods exhibit imbalanced behavior: the z score-based method achieves very high precision but extremely low recall, missing most anomalies, while manual percentage change-based detection achieves high recall but very low precision, producing excessive false positives. The agentic system maintains a better balance between precision and recall compared to all baselines.

B. Quantitative Comparison and Statistical Validation

Full comparison table

Synthetic spikes vs fixed baselines.

Precision / recall apply only to *binary anomaly detection* (first four rows). **Root-cause** rows report *accuracy* (top-1 / top-3 hit rate vs injected key); **decision_rank_consistency** is *Kendall* τ vs impact ordering — those are not precision/recall, so those cells show —.

System	Precision	Recall	F1 / score
agentic_system	0.7500	0.3750	0.5000
baseline_1_static_threshold	0.1111	0.3750	0.1714
baseline_2_z_only	1.0000	0.1250	0.2222
baseline_3_manual_script	0.0897	0.8750	0.1628
root_cause_top1	—	—	0.1250
root_cause_top3	—	—	0.1250
decision_rank_consistency	—	—	0.7576

Fig. 7. Quantitative comparison between the proposed system and baseline approaches.

Fig. 7 summarizes the quantitative performance of the agentic system compared to baseline approaches. The agentic system achieved a precision of 0.75, recall of 0.375, and F1 score of 0.50, outperforming all baseline methods. Static thresholding achieved an F1 score of 0.17, z score-based detection achieved 0.22, and manual percentage change-based detection achieved 0.16.

Across 5 runs with independent random seeds, the agentic system showed a mean F1 score of 0.46, compared to 0.19 for the static threshold baseline. A paired t test yielded a p value of 0.0004, indicating that the improvement is statistically significant. All experiments were conducted with fixed random

seeds to ensure reproducibility, and paired statistical tests were used to compare matched runs across the same injection configurations.

Additionally, the system achieved a Kendall tau ranking correlation of 0.7576, demonstrating consistency in ranking recommended actions.

C. Anomaly Detection Behavior

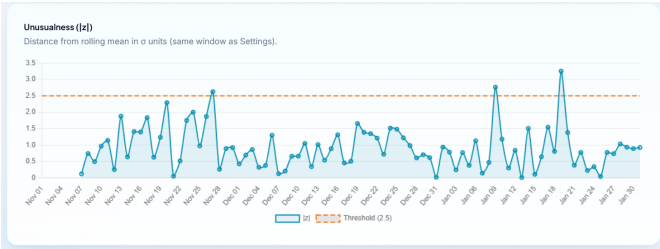

Fig. 8. Detection of anomalies based on deviation from rolling baseline.

The system computes unusualness as the absolute z score relative to a rolling mean and standard deviation. Data points exceeding the threshold of 2.0 under the balanced sensitivity configuration are flagged as anomalies. This approach enables adaptive detection by continuously updating statistical baselines, allowing the system to respond to shifts in data distribution while maintaining sensitivity to significant deviations.

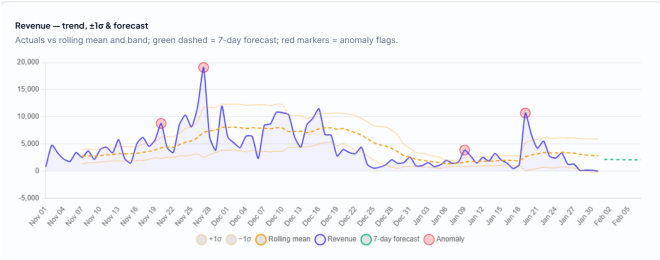

Fig. 9. Revenue trend with rolling mean, deviation bands, and detected anomalies.

The revenue trend visualization shows actual values alongside the rolling mean and deviation bands. Anomalies correspond to points where the observed value deviates significantly from the expected range. The system correctly identifies major spikes and drops in revenue while maintaining stability during normal periods. The use of rolling baselines allows detection to adapt over time, avoiding the limitations of fixed threshold-based methods.

D. Root Cause and Decision Effectiveness

Fig. 10 shows the contribution of different dimensions to the detected anomaly for a selected focus day. The system ranks dimensions based on their contribution scores, enabling identification of key drivers such as product tier and region.

Root cause identification achieved 12.5% top-1 accuracy and 12.5% top-3 accuracy on injected anomaly days. The

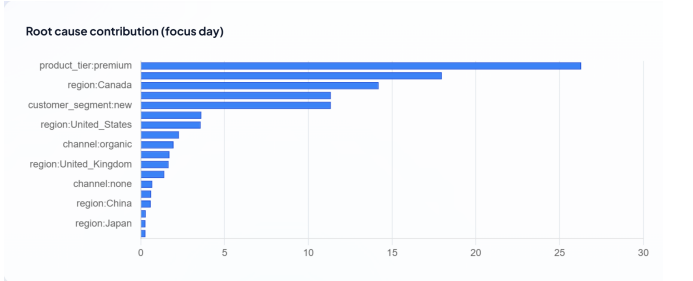

Fig. 10. Contribution of dimensions to detected anomaly.

relatively low accuracy reflects the synthetic nature of dimensional proxies in the GA4 dataset, where true dimension level contributions are approximated rather than directly observed.

The Decision Agent generates ranked corrective actions based on identified causes. The mean top-1 utility of the agentic system was 0.47, compared to 0.43 for random action selection, representing a gain of 0.04.

E. System Efficiency

The system processes data sequentially with an average end to end latency of 25.67 ms per day, including dashboard rendering and visualization overhead.

The underlying pipeline computation without visualization incurs an average latency of 6.94 ms per day, indicating that most overhead arises from the user interface layer.

The use of shared memory reduces redundant computations by allowing agents to reuse intermediate results such as rolling statistics, anomaly scores, and root cause outputs, enabling efficient execution suitable for near real time applications.

F. Explanation Quality

Two independent reviewers evaluated anomaly day explanations using a structured rubric covering three criteria: factual consistency with computed outputs, completeness of root cause coverage, and clarity of recommended actions. Each criterion was rated on a 1 to 5 scale. Inter rater agreement was measured using Cohen's Kappa to validate scoring reliability. The Cohen's Kappa score will be reported upon completion of the reviewer evaluation.

G. Observations and Interpretation

The results demonstrate that the agentic architecture improves detection performance and decision quality compared to baseline approaches. The integration of multiple agents enables structured reasoning across monitoring, anomaly detection, root cause analysis, and decision generation, while shared memory ensures consistency of intermediate states across all stages.

The system achieves higher precision compared to baseline methods, reducing false positives, while maintaining sufficient recall to detect meaningful anomalies. This balance is reflected in the improved F1 score observed in the evaluation.

The system produces interpretable outputs through explicit root cause attribution and ranked corrective actions, enabling traceability from detection to recommendation.

These results confirm that the system meets the objective of accurate anomaly detection, interpretable root cause analysis, and actionable decision support.

VI. Discussion

The results demonstrate that the agentic architecture enables consistent improvement over baseline methods by combining statistical detection with structured multi agent reasoning. Unlike single stage approaches, the system maintains continuity across monitoring, anomaly detection, root cause analysis, and decision generation through a shared state, allowing each stage to build on prior outputs rather than operating independently.

The system exhibits autonomous behavior through sequential execution of agents coordinated by the central controller. At each time step, the monitoring agent observes incoming data, anomaly detection evaluates deviation, root cause analysis attributes contributing factors, and the decision agent generates ranked actions without manual intervention. The controller governs progression and re evaluation based on intermediate outputs, ensuring that inconsistent or low confidence states do not propagate further.

The system qualifies as an agentic architecture through four properties. First, all agents coordinate through shared state rather than simple linear output passing. Second, the Central Controller acts as a planning layer that evaluates confidence thresholds and validates intermediate outputs before allowing stage progression. The controller interacts with every agent in the pipeline: it triggers the monitoring agent to begin each cycle, gates anomaly detection outputs before they reach root cause analysis, validates attribution results before decision ranking, and ensures explanation outputs are consistent with upstream computations. Third, the re-evaluation mechanism enables the system to revise its own decisions when inconsistencies are detected, rather than executing a fixed forward pass. Fourth, the system maintains controlled autonomy by reasoning independently through detection, attribution, and decision ranking while deferring final action execution to human approval. These properties distinguish the architecture from a conventional sequential pipeline where each stage executes independently without conditional control or shared coordination.

The results indicate that autonomy is not limited to detection but extends to decision support, where actions are derived directly from identified causes. This reduces reliance on manual analysis and enables faster response to anomalies while maintaining traceability between observed deviation and recommended actions.

Several limitations were observed. Root cause accuracy remains low due to the use of synthetic dimensional proxies in the dataset, where true contribution signals are not directly observable. This affects the ability to precisely identify causal drivers. Additionally, the system relies on statistical assumptions such as rolling mean and variance, which may not capture complex temporal dependencies in highly non-stationary data.

These limitations are partially mitigated through the use of multiple signals, including EWMA based smoothing and confidence scoring, which improve robustness under noisy conditions. The inclusion of human approval before action execution also prevents propagation of incorrect decisions in uncertain scenarios.

Overall, the system demonstrates that combining statistical methods with agent-based orchestration provides a scalable and interpretable approach to anomaly detection and decision support, while highlighting areas for improvement in causal attribution and modeling complexity.

VII. Conclusion and Future Work

This work presents an agentic multi agent system for anomaly detection, root cause analysis, and decision support over time series data. The system integrates monitoring, anomaly detection, attribution, and action generation through a shared-memory architecture coordinated by a central controller. Experimental results show that the proposed approach achieves an F1 score of 0.50 under balanced sensitivity configuration, outperforming baseline methods while maintaining a better balance between precision and recall. The system also demonstrates consistent action ranking with a Kendall tau correlation of 0.7576 and provides interpretable outputs linking detected anomalies to root causes and recommended actions.

A key contribution of this work is the use of coordinated agents with shared state, enabling continuity across stages and reducing redundant computation. The system operates autonomously through a structured think act observe cycle, while maintaining safety through confidence gating and human approval before execution of actions.

Several limitations were identified. Root cause accuracy remains limited due to the use of synthetic dimensional proxies, and the reliance on rolling statistical methods may not fully capture complex temporal patterns in highly dynamic data. Additionally, evaluation of explanation quality is ongoing and requires further validation through human assessment.

Future work will focus on improving root cause attribution using causal inference or learned attribution models, incorporating advanced temporal models such as sequence based neural networks, and expanding tool integration for real world action execution. Further evaluation across multiple datasets and real world deployment scenarios will also be conducted to validate generalizability and robustness.

References

[1] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.

[2] A. Laptev, S. Amizadeh, and I. Flint, "Generic and Scalable Framework for Automated Time-Series Anomaly Detection," in *Proc. ACM SIGKDD*, 2015.

[3] S. Vallis, J. Hochenbaum, and A. Kejariwal, "A Novel Technique for Long-Term Anomaly Detection in the Cloud," in *Proc. USENIX*, 2014.

[4] J. Pearl, *Causality: Models, Reasoning, and Inference*. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[5] M. Wooldridge, *An Introduction to MultiAgent Systems*. Hoboken, NJ, USA: Wiley, 2009.

[6] D. Gunning and D. Aha, "DARPA's Explainable Artificial Intelligence Program," *AI Magazine*, vol. 40, no. 2, pp. 44–58, 2019.

[7] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, 2022.

[8] T. Schick *et al.*, "Toolformer: Language Models Can Teach Themselves to Use Tools," arXiv:2302.04761, 2023.
[9] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.